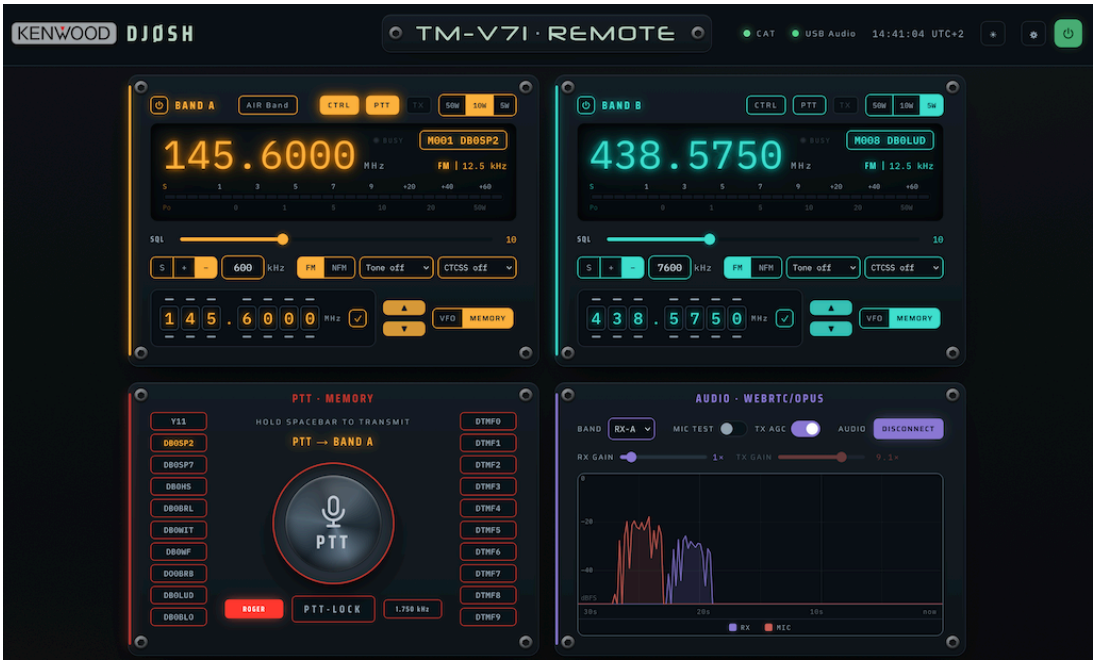
TM-V71 Remote

User & Technical Manual

Kenwood TM-V71 web remote

Version 3.2



Kenwood TM-V71(A/E) web remote + WebRTC audio + HackRF panadapter, for a Raspberry Pi.

1 Overview

TM-V71 Remote is a modern, dependency-light web remote control for the Kenwood TM-V71(A/E) dual-band FM transceiver, built around a direct serial driver. It gives full radio control in the browser, two-way browser audio over WebRTC/Opus, complete memory-channel management, an optional HackRF panadapter, classic 5-tone selective calling, a CW/RTTY/POCSAG digimodes decoder/encoder, a raw RX recorder, and optional off-air callsign recognition (Vosk). It installs as a Progressive Web App (PWA) and is designed to run on a Raspberry Pi.

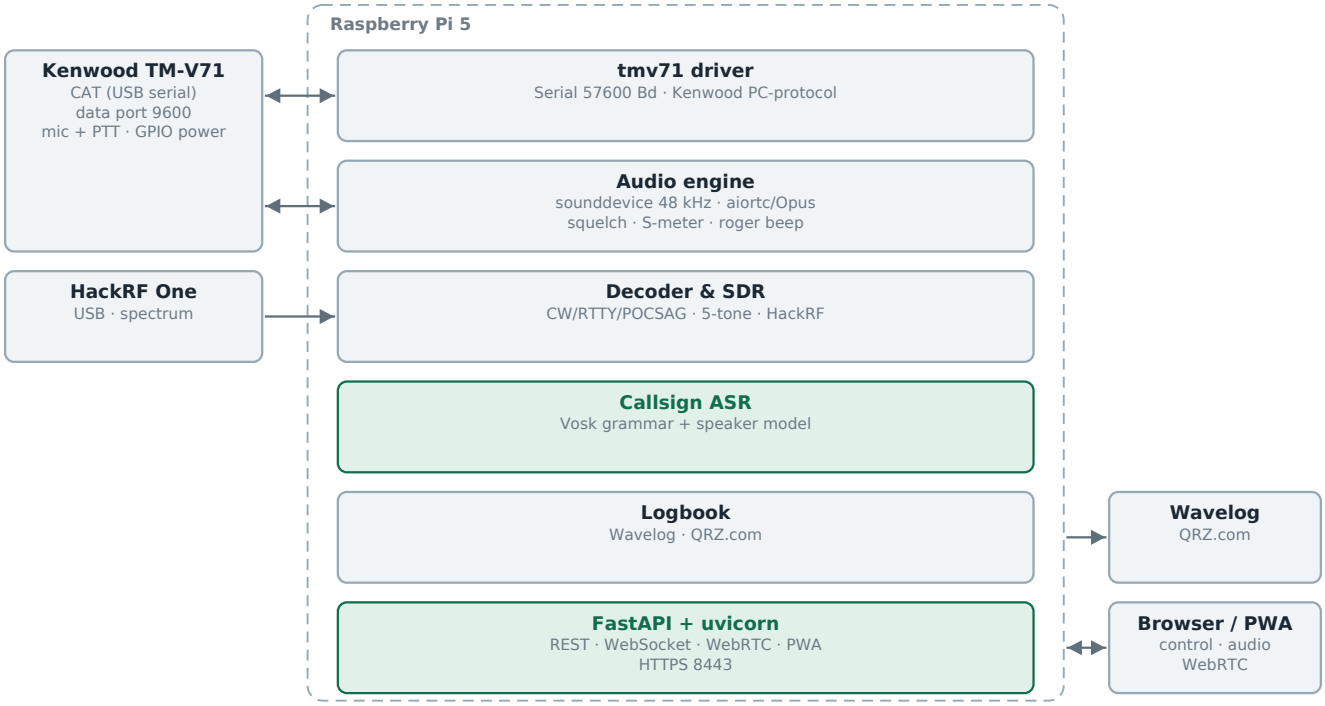
Unlike hamlib, whose TM-V71 backends are unreliable, this project speaks the radio's documented PC command set directly and exposes the radio's full feature set, including per-channel memory programming.

2 Features

- Full live control of both bands (A/B): frequency, VFO/memory mode, repeater shift & offset, CTCSS/DCS, step, control band, and PTT over CAT.
- Memory channels (CHIRP-level): read, write, delete, rename any of the 1000 channels, plus CSV import/export.
- Live status pushed to the browser over a WebSocket; transmit lights the UI.
- Two-way audio: direct WebRTC/Opus between browser and backend via aiortc; the mic feeds the radio only while PTT is engaged.
- Optional HackRF One waterfall: real-time panadapter (auto-following the tuned frequency) or a wideband sweep.
- Classic 5-tone selective calling (ZVEI-1/2, CCIR, EEA): call, decode, and mute RX until your own ID is received.
- CW (Morse), RTTY (Baudot/AFSK) and POCSAG paging (512/1200/2400 baud, numeric + alphanumeric, BCH FEC) decode + encode over the FM audio path; CW auto mode tracks the received speed and tone pitch, plus a button to decode a captured RX buffer off-line.
- Audio processing: RX de-emphasis (for a flat 9600/discriminator feed), BUSY-gated software squelch, TX AGC, and voice low-pass filters.
- Raw RX recorder with WAV download (e.g. to build ASR training data).
- Off-air callsign recognition (optional, Vosk): detects spoken German callsigns, verifies them against the BNetzA list (name/town/class, or VOID if unassigned), shown in the title bar and a toast.
- Installable PWA with a mobile landscape swipe-deck layout.
- Resilient operation: the phone screen is kept awake, browser audio auto-reconnects after a network glitch, and a backend watchdog releases a latched PTT if every client disappears.
- GPIO power switch, auto power-off, TX power, squelch, in-display S-meter.
- Two themes (dark/light); no build step for the UI.

3 Architecture

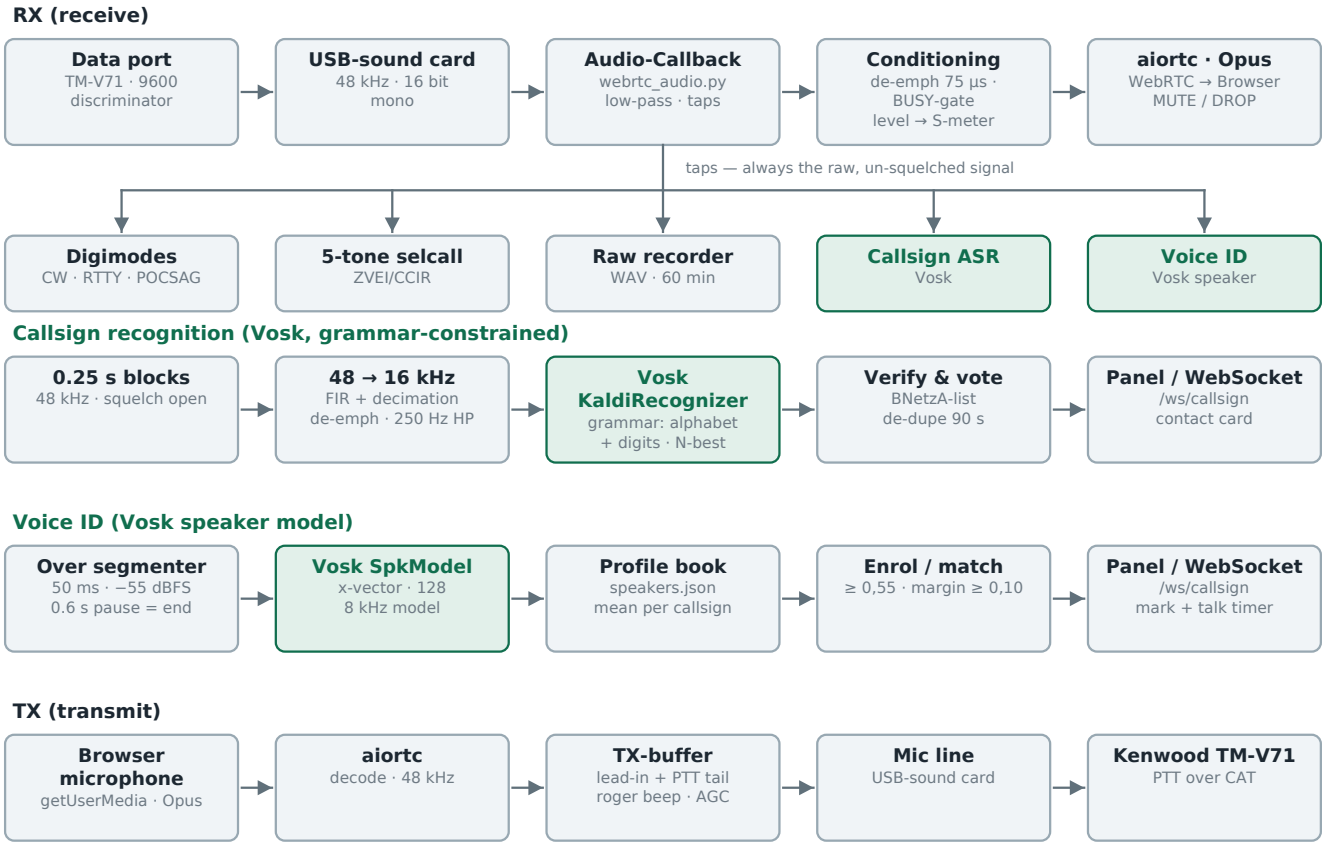
Everything below runs on the Pi. The radio is reached over two independent paths — CAT commands on the serial line, and audio on the data port through a USB sound card — and the browser sees only the one HTTPS address the backend serves.



Everything runs on the Pi; the browser gets control, status and audio over one HTTPS address.

Audio path

One capture callback owns the received audio. What the browser hears is conditioned and squelched; every decoder is fed from a tap taken BEFORE that, on the raw signal, which is why the S-meter, the recorder and both Vosk chains keep working while MUTE or DROP is on. Vosk appears twice on purpose: the grammar-constrained decoder reads spoken callsigns, and a second, plain recognizer with the speaker model turns a whole over into a voiceprint (the grammar decoder runs with N-best alternatives, and in that mode Vosk returns no x-vector).



The decoders always sit on the raw signal — squelch, MUTE and DROP only affect what the browser hears.

The backend owns the serial port directly (backend/app/tmv71.py). One FastAPI process serves the SPA/PWA, the REST control endpoints, the live-status WebSocket, and the WebRTC audio signalling — no extra services. Audio is 48 kHz / 16-bit / mono internally (Opus' native rate).

4 Requirements & Hardware

- Raspberry Pi (tested on Debian 13 / aarch64), Python 3.11+.
- Kenwood TM-V71(A/E) on a serial port (FTDI programming cable), 57600 baud.
- A USB sound interface wired to the radio (data port or mic/speaker), full-duplex.
- System packages: portaudio19-dev, swig + liblgpio-dev (optional GPIO).
- Optional: a HackRF One plus the hackrf host tools for the waterfall.
- Optional: vosk + the small German model (offline callsign recognition), and pypdf + the BNetzA Rufzeichenliste PDF (name/town/class + VOID check).

5 Installation

```
sudo apt-get install -y portaudio19-dev python3-venv swig liblgpio-dev
git clone https://github.com/CQ-DJ0SH/tmv71-remote.git
cd tmv71-remote/backend
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

For a reboot-proof setup, install the systemd unit from the deploy/ directory. The service runs uvicorn with TLS on port 8443.

6 Running over HTTPS

Browser microphone access (getUserMedia) and the PWA service worker require a secure context, so the server runs over HTTPS. A quick self-signed certificate is enough on the desktop (accept the warning once); for installing the PWA on a phone you need a trusted certificate (see chapter 9).

```
cd backend && mkdir -p certs
openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
  -keyout certs/key.pem -out certs/cert.pem -subj "/CN=tmv71-remote" \
  -addext "subjectAltName=IP:<pi-ip>,DNS:localhost,IP:127.0.0.1"
.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8443 \
  --ssl-keyfile certs/key.pem --ssl-certfile certs/cert.pem
```

Open https://<pi-ip>:8443/ and accept the certificate once.

7 The Web Interface

Band panels (VFO A / VFO B)

Each band shows the frequency on a 7-segment display with two stacked meters under a shared S-scale: a real S-meter (S0–S9) in the active band's colour, and below it the AF level / mic-modulation bar (1 s peak-hold). Plus controls for VFO/memory mode, CTRL/PTT band selection, TX power, squelch (remembered per band across power cycles), repeater shift/offset, tone and bandwidth. The digit tuner lets you click bars above/below each digit to step the frequency; AIR Band tunes band A to the 118–137 MHz air band (receive-only).

The S-meter is derived from FM quieting: the high-band noise on the flat RX is inverse to signal strength (loud hiss = no signal, full quieting = strong signal), so it estimates the received signal even though the TM-V71 sends no numeric RSSI over CAT — only a binary BUSY status. It is a relative quieting estimate: a readable signal reads mid/upper scale, a strong local signal saturates at S9, and it snaps back to S0 the moment the carrier drops.

PTT & memory quick keys

Hold the large PTT button (or the space bar) to transmit; PTT-LOCK latches transmit. PTT and PTT-LOCK require connected audio (there is no mic otherwise) — they are disabled while audio is off and released automatically if audio disconnects. ROGER adds a descending three-tone beep (d6 · a5 · e5, i.e. 1175/880/659 Hz, 60 ms each with 30 ms gaps) on release, at the level set in the audio settings; while transmitting the button shows a count-up timer (MM:SS). The 1750 Hz button arms a tone-call. Memory quick keys recall channels 0–16 (M0–M9 in the left column, M10–M16 in the right; the loaded channel's key glows); below them the right column sends three DTMF memories (0–2). A status line shows per-band BUSY, the ASR state and the live RX/TX gain (in the PWA; the desktop shows the transmit hint). On mobile, mini RX/TX VU bars with peak-hold flank the button.

MUTE and DROP

Two buttons flank the “PTT → BAND x” caption and silence the received audio, differing only in when they stop:

- MUTE (left) is the plain one: it stays until it is pressed again. The label reads MUTED while it is on.
- DROP (right) blanks the transmission in progress and lifts itself as soon as that station stops keying — for an interferer or a rag-chew you do not want to hear, without having to remember to un-mute afterwards.

Both glow in the colour of the transmit band while armed. Three details are worth knowing:

- DROP releases on the falling edge of that band's BUSY signal, not merely because the channel is quiet: arming it on an idle channel keeps the audio muted until the end of the next transmission, rather than un-muting again on the next status update.
- DROP also lifts after three seconds without speech, even while BUSY is still up — the permanent carrier of a repeater would otherwise keep the audio muted indefinitely. The threshold is on the AF level, so a quiet carrier counts as silence while an over does not.
- Muting is done in the browser, on the audio element. The S-meter, the raw RX recorder and the callsign recognition keep receiving the signal — only what you hear is silenced.
- A three-tone chime confirms each change (e5 · a5 · d6 rising on release, the same triad

reversed when muting), so the two are told apart without looking. It is generated in the browser and can therefore never be mixed into the transmitted audio, unlike the roger beep, which deliberately is.

Audio (WebRTC/Opus)

Open the AUDIO panel, pick the RX band with the RX-A/RX-B switch, click CONNECT and allow the microphone. RX and mic levels are shown live, with the WebRTC RX/TX data rate in the graph corner. Controls: RX/TX gain (with a recommended-default tick), MIC (mic test — meters the mic without keying, records while on and replays your audio over RX when switched off; RX is muted during the test), AGC (automatic TX level), and a small recorder — ● REC / ▶ PLAY plus a WAV download of the raw, un-squelched RX feed (up to 60 min; e.g. to build ASR training data; the downloaded file is named with the date and time it was saved). TX timing (buffer / trail) and the USB card mixer are in Settings > Audio. The link auto-reconnects after a network glitch and is restored on the next launch.

RX conditioning (Settings > Audio): a RX de-emphasis (adjustable time constant, on by default) restores natural voice tone when the audio comes from a flat discriminator / 9600-baud data output; a fixed ~180 Hz high-pass on the listen path removes the CTCSS/PL sub-audible tone (67–254 Hz) and DC hum that this flat output passes and de-emphasis would otherwise lift (audible as a low speaker hum), while leaving voice untouched; a BUSY-gated software squelch re-applies muting from the radio's own busy status for that always-open output; and TX/RX voice low-pass filters (≤ 3.5 kHz) tame hiss. The decoders always receive the un-squelched, un-filtered signal.

Bluetooth headsets: transmit audio is captured from the phone's built-in microphone (not the headset's), so the headset stays on the A2DP profile and receive audio keeps coming through in good quality. Using the headset mic would force Android onto the mono HFP/SCO profile and, on many phones, leave RX stuck until Bluetooth is toggled.

HackRF waterfall

If a HackRF One is connected, this panel shows a live spectrum stacked over a waterfall: a panadapter centred on the tuned frequency (auto-following) or a wideband sweep. Receive-only; LNA/VGA gains and a display level are adjustable.

Selcall (classic 5-tone)

Send and decode classic selective calls (ZVEI-1/2, CCIR, EEA). Enter a 5-digit CALL code and press CALL (keys PTT). Enter your own ID and press MUTE to silence RX until your ID is received — then it un-mutes automatically. Over FM this is AFSK; use a dummy load when setting up.

Digimodes (CW / RTTY / POCSAG)

Switch between CW (Morse), RTTY (Baudot/AFSK) and POCSAG paging. DECODE shows received text; type into the field and SEND to transmit (keys PTT); the CW text input is forced upper case. Parameters: CW WPM/pitch — with an AUTO mode that tracks both the received speed and tone pitch (shown live on the sliders) and rejects voice/noise so it locks onto the CW even after a spoken ident; RTTY baud/shift/mark; and POCSAG baud (512/1200/2400), RIC, function and numeric/alphanumeric (auto-detected on RX) — e.g. monitor DAPNET on 439.9875 MHz with per-page RIC/FUNC/timestamp output. The REC

button decodes the raw RX recorder buffer off-line in the current mode. Over the FM radio this is MCW / AFSK / FSK — not native HF modes.

Callsign recognition (Vosk)

An optional, offline speech-recognition pass on the RX audio that detects spoken German callsigns; enable it in Settings > Audio. A grammar-constrained Vosk model (ITU/NATO phonetic alphabet plus German digits — the German spelling alphabet and letter names were dropped as their short, homophone-prone words caused most false matches) stays usable on noisy FM voice; the recognised letters are assembled into a callsign, restricted to the real German BNetzA allocation blocks (always 5–6 characters) and verified against the BNetzA Rufzeichenliste. Accuracy is raised by N-best rescoring — Vosk returns several hypotheses per over and the best callsign across them is chosen, preferring an assigned one — and by repetition voting: a listed call is shown at once, while an unassigned (VOID) hit must be heard twice within a short window before it appears, suppressing one-off mishears (operators send their call 2–3× anyway). A hit appears in a framed field in the title bar (coloured to the RX band) and as a toast, enriched from the offline list with the holder's name, town and licence class (A/E/N); a call that is not assigned is still shown but flagged VOID. Your own callsign is ignored, it runs only while the squelch is open, and it can also grade the mic-test audio. The callsign list is built once from the PDF with a converter (python -m app.callsign_list); QRZ.com is used only for a manual lookup in the logbook, never by the ASR.

Powering the radio down suspends the recognition — there is no RX audio to analyse — and switching it back on resumes it. Your setting is not overwritten by this, so detection does not quietly stay off after a power cycle. The ASR indicator and the detected-callsign field stay on screen throughout; only the LED changes: green and pulsing while listening, red while suspended, dim when switched off. The pulse is what says “listening right now”, so neither off-state pulses.

Band scan

Sweep a VHF/UHF range or the memory bank and see an occupancy spectrum + waterfall. Double-click a channel to tune the control VFO to it.

ASR contacts

A panel below the band scan that collects every recognised station as an index card, so the last overs are readable at a glance instead of as a scrolling log. Cards sit in a tray of empty slots, newest first. The last 200 entries are kept on the Pi and restored when the panel opens; CLEAR ALL empties the view. In the mobile deck the panel has no tab — swipe to it past the band scan.

Each card carries:

- The callsign, in the largest and boldest type on the card — it is the identity of the contact. A zero is drawn slashed (DJØSH) so it cannot be misread as the letter O; that is display only, everything sent to the logbook keeps the plain 0.
- An avatar whose letters and colour are derived from the callsign itself (the characters after the region digit, plus a hue hashed from the whole call), so a station looks the same in every session without anything being stored.
- All three German licence classes A / E / N, with the holder's own one lit and the other

two dimmed. If none is lit, the call is not in the BNetzA list.
- Name and town of the holder from the offline BNetzA list — never from QRZ.com, which the ASR does not query.
- The time the station was last heard and its talk timer, pinned to the bottom edge of the card.

A station heard again does not get a second card. The existing one flashes, is marked, and counts up (×2, ×3 …) — but it stays where it is, so the tray does not reshuffle under you on every over. Exactly one card carries the red border at a time: the most recently heard. The ordering therefore follows first contact, not last mention.

Hovering a card shows the recogniser detail that used to fill the log lines: word confidence (0.00–1.00, the mean over the individual spelled letters — a statement about the acoustics, not about whether the callsign is real), the S-value and receiving band at the moment of detection, the raw words Vosk actually heard, the rejected N-best candidates, and any 5/6-character correction that was applied.

Two buttons per card: the play symbol logs the QSO straight to Wavelog with the name from the BNetzA list pre-filled, and turns teal once it has gone through so nothing is sent twice. The cross removes a misrecognised contact. That deletion happens on the Pi, not just in the browser: the card is dropped from the log buffer (otherwise it would return the next time the panel opens), every connected client loses it at once, and the callsign is released from the 90-second de-dupe window so a corrected reading can be reported immediately.

Talk time. The clock face in the bottom row of a card is the start/stop button for that station's timer — and the count also runs by itself: it starts on the rising edge of BUSY and stops on the falling one, so an over is timed without touching anything. The clock always belongs to exactly one card, the marked one. Clicking a card moves the mark, and with it the timer, onto that contact — for when the recognition attributed an over to the wrong station. Every card keeps its own total: switching the focus banks the running stretch on the card it came from and picks up the new card's figure, so nothing is lost or counted twice. The display is fixed at hh:mm:ss, otherwise the button would change width as it counts.

The panel head carries the sum of all timers (Σ) and five controls:

- The input field takes a callsign by hand when the recognition misses one — upper-case letters and digits only, filtered while typing. LOG (or Enter) creates the card through the backend exactly like a recognised one, so it lands in the history and reaches every connected client. A message says whether the BNetzA list knew the call.
- MOD marks the focused card as net control. Its timer keeps running and stays readable on the card, but it is left out of every figure: neither the Σ total nor the ranking counts it, and it does not set the scale of the bars — without that, the station holding the round together would flatten every other bar. A flagged card wears a ring around its avatar. The flag lives in the browser (local storage), not on the Pi: it belongs to whoever is listening to this round, and it survives a reload.
- STATS opens the evaluation: every card by talk time, longest first, with callsign, name, a bar and the time. The bar is scaled against the longest over rather than against the sum. Moderators are listed below the ranking, without a bar and without a rank. COPY puts the list on the clipboard as plain text. The dialog keeps counting while a station is being heard, so it can stay open through an over.
- CLEAR ALL empties the tray — in dark red, because it takes every card at once. It clears the view only; the cross on a single card also deletes that contact on the Pi.

- VOICE, at the right-hand end, switches the speaker recognition on and off (see below). Its lamp pulses green while it is listening; an outlined button means it only observes, a filled one that it acts.

Correcting a callsign: click the call on the card and type over it (Enter applies, Escape and clicking away cancel). The card keeps its talk time, its repeat count and its place, name and town are re-read for the corrected call, and the change reaches every open client. If the corrected call already has a card, the two are merged and their times added. The voice profile is renamed with it — otherwise a mis-heard call would keep collecting a voiceprint under a name that never existed while the real station never gets one.

Voice ID — an over without a callsign

Not every over carries a spoken callsign, and the talk timer then counts onto whoever was marked last. Speaker recognition closes that gap: the voiceprint of an over is compared against the stations whose callsign HAS been heard. Enrolment costs nothing — every recognised call labels its own audio, so the profiles build themselves while the panel is simply used.

- An over ends at the PAUSE in speech, not at the carrier. Through a repeater BUSY stays up for the whole QSO, so the falling edge never comes and every station would land in one segment with their voices averaged together. The pause boundary works in simplex too, where the carrier drop simply arrives first.
- Under about 3 s of speech nothing is guessed at: the over is skipped. An over in which two callsigns were heard is skipped as well, because labelling it would spoil both profiles.
- Two stages. Observe only decides and reports — the verdict appears above the card tray and in the card's hover detail, and nothing moves. Assign additionally moves the mark and the talk timer onto the recognised station; such a card is drawn with a dashed border so it is never confused with the solid red of “just heard saying its call”. The stage is chosen in Settings > Audio > Voice ID.
- A card picked BY HAND always wins: while a manual selection holds, the voice moves neither the mark nor the timer. It is released when the next over begins, so the correction applies to the over it was made in. A callsign actually heard still moves the mark — that is evidence, not a guess.
- It needs the callsign recognition switched on (it listens to the same audio tap), and it costs about 1.3 s of one core per over, afterwards.

Measured on two off-air recordings of the same round (14 + 11 overs, four stations in both): the same station across recordings scored 0.62–0.79 cosine, different stations 0.33 on average and 0.64 at worst. At a threshold of 0.55 with a 0.05 margin over the runner-up, 3 of the 4 known stations were identified, none wrongly, and none of the 7 stations unknown to the profile set was mistaken for a known one. The margin does the real work: a stranger's best match is flat (0.01–0.08 ahead of the second), a true match stands clear (0.20–0.34). Treat it as an assistant that may abstain, not as proof — and never log a QSO on a voice match alone.

The model is Vosk's own speaker model (vosk-model-spk-0.4, ~14 MB), an x-vector trained on 8 kHz telephone speech — narrowband, noisy, channel-varied, which is far closer to FM off a repeater than a wideband model would be. A voiceprint is biometric data: the profiles stay on the Pi in speakers.json (gitignored, never uploaded) and a profile is dropped with its

card.

Logbook (Wavelog + QRZ.com)

Logs QSOs to a locally installed Wavelog instance. Enter just the callsign (and optionally a name) — frequency, band, mode, date/time and your own callsign are filled in automatically from the live control band and station profile. LOOKUP fetches the operator's name, grid, QTH, country and e-mail from QRZ.com (XML data API) and 'worked before' / DXCC from Wavelog. LOG QSO sends the contact as ADIF. A green dot shows when Wavelog is reachable; the panel lists the most recent QSOs (with the looked-up details, deletable individually or via CLEAR) and Wavelog's QSO counts (today/month/year/total). Configure the Wavelog URL, API token and station profile, and the QRZ.com username/password, in Settings > Logging. Credentials are stored only on the Pi (runtime.json), never committed.

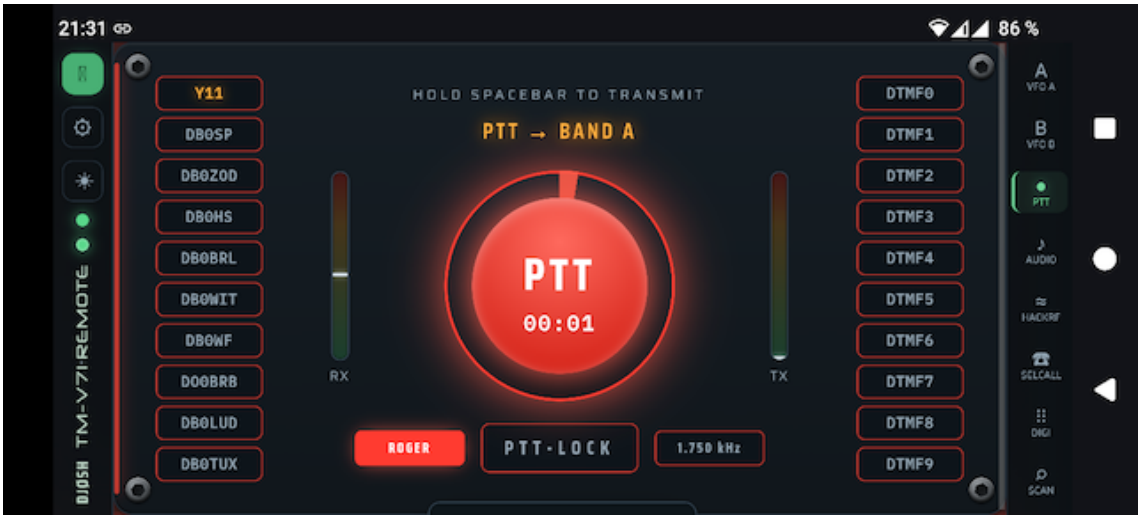
Settings

Tabs: General (callsign, API backend URL, serial port/baud, GPIO power, auto power-off, logo, GitHub self-update, Root-CA download), Audio (device, USB mixer, voice filters, test tone, TX timing), Rig-Info, Rig-Memory, Rig-DTMF, Logging (Wavelog + QRZ.com), and Pi-Hardware (host metrics).

8 Mobile App (PWA)

The UI installs as a Progressive Web App: full-screen, with an app-shell service worker for instant launch. On phones the panels become a vertical swipe deck — swipe up/down, one panel per screen (this keeps the deck's scroll axis off the horizontal sliders, so they stay usable) — with the title bar as a slim vertical strip on the left and an icon tab rail on the right. Past the last panel is an info page listing the app version and browser/environment details. The app is forced to landscape; portrait shows a rotate hint. Install via the browser menu (Install / Add to Home Screen); on iOS use Safari > Share.

While the app is open the phone screen is kept awake via the Screen Wake Lock API, so it won't dim or lock mid-QSO. The browser audio link reconnects automatically after a brief network interruption, and if the connection to the backend is lost while transmit is latched, the PTT is released (locally and by a backend watchdog) so the rig can never stay keyed unattended.



9 Trusted Certificate (Root CA)

A self-signed certificate is fine on the desktop, but mobile browsers will not install the PWA or run the service worker without a trusted certificate. Create your own root CA, sign the server certificate with it, and trust the CA on the phone. A download link for the CA appears in Settings > General once it exists.

```
cd backend/certs && mkdir -p ca
# 1) Root CA (10 years) — keep ca.key secret
openssl genrsa -out ca/ca.key 4096
openssl req -x509 -new -key ca/ca.key -sha256 -days 3650 \
  -subj "/CN=TM-V71 Remote Root CA" \
  -addext "basicConstraints=critical,CA:TRUE,pathlen:0" \
  -addext "keyUsage=critical,keyCertSign,cRLSign" -out ca/ca.crt
# 2) server cert signed by the CA (list every name/IP in the SAN)
openssl genrsa -out key.pem 2048
openssl req -new -key key.pem -subj "/CN=tm-v71.example.lan" -out s.csr
openssl x509 -req -in s.csr -CA ca/ca.crt -CAkey ca/ca.key \
  -CAcreateserial -days 825 -sha256 -extfile leaf.ext -out cert.pem
sudo systemctl restart tmv71-remote.service
```

Install ca/ca.crt on the phone (Settings > Security > Install a certificate > CA certificate). The leaf is valid 825 days; re-issue it from the same CA without re-installing on phones. Keep ca/ca.key secret.

10 Configuration

Settings are read from environment variables (prefix TMV71_) or a .env file, with web-UI changes persisted to backend/app/runtime.json. Key variables:

```
TMV71_SERIAL_PORT=/dev/ttyUSB0   TMV71_SERIAL_BAUD=57600
TMV71_HOST=0.0.0.0               TMV71_PORT=8443
TMV71_AUDIO_DEVICE=NAD           TMV71_AUDIO_ENABLED=true
TMV71_GPIO_POWER_PIN=17          TMV71_CALLSIGN=DJ0SH
TMV71_ASR_MODEL_DIR=...          TMV71_ASR_CALLLIST_PDF=...
TMV71_SSL_CERTFILE=...  TMV71_SSL_KEYFILE=...
```

11 REST & WebSocket API

Control & status

```
GET  /api/status            live radio state
POST /api/frequency         set VFO frequency
POST /api/band-mode         VFO / memory / call
POST /api/control-band      select control band
POST /api/ptt               key / un-key (CAT)
POST /api/ptt-band          select TX band
POST /api/squelch /step     squelch level / step
POST /api/vfo               shift/offset/tone/bw
GET  /api/info /version     rig + app info
WS   /ws                    live status stream
```

Memory channels

```
GET    /api/memories?start&end   list channels
GET    /api/memories/{ch}        one channel
PUT    /api/memories/{ch}        write channel
DELETE /api/memories/{ch}        clear channel
GET    /api/memories.csv         export CSV
POST   /api/memories/import      import CSV
POST   /api/recall               recall to band
```

Audio

```
POST /api/webrtc/offer      WebRTC SDP offer
GET  /api/audio/status      RX/TX levels, flags
GET  /api/audio/devices     list sound devices
POST /api/audio/device      pick device
POST /api/audio/gain        rx/tx gain + TX AGC
POST /api/audio/buffer      tx buffer / ptt tail
POST /api/audio/tones       roger/test/mic/lowpass/de-emph/squelch
POST /api/audio/record[/clear]  raw RX recorder
GET  /api/audio/record.wav  download recording (WAV)
GET/POST /api/audio/mixer   USB card mixer
```

Digimodes & selcall

```
GET  /api/digi              CW/RTTY/POCSAG status
POST /api/digi/config       mode + parameters
POST /api/digi/tx           encode + transmit
POST /api/digi/decode-recording  decode the RX buffer
WS   /ws/digi               decoded text stream
GET/POST /api/asr/config    callsign recognition (Vosk)
POST /api/asr/log           add a callsign by hand
PATCH /api/asr/log/{call}   correct a callsign (profile follows)
GET/POST /api/asr/speaker   voice ID: on/off, stage, limits
DELETE /api/asr/log/{call}  drop a misrecognised contact
WS   /ws/callsign           recognised-callsign events
GET  /api/selcall           5-tone status
POST /api/selcall/config    standard / tone / own
POST /api/selcall/tx        send a 5-tone call
```

```
WS   /ws/selcall            decoded calls stream
```

SDR & scan

```
GET  /api/hackrf            SDR status
POST /api/hackrf/start|stop|config
WS   /ws/hackrf             spectrum/waterfall frames
GET  /api/scan  POST /api/scan/start|stop  band scan
```

Logbook

```
GET/POST /api/log/config     logbook credentials (Wavelog + QRZ)
POST /api/log/test           test the Wavelog connection
POST /api/log/qrz/test       test the QRZ.com login
GET  /api/log/stations       Wavelog station profiles
POST /api/log/lookup         callsign lookup (QRZ + Wavelog)
POST /api/log/qso            log a QSO
GET  /api/log/recent         recent QSOs + online + Wavelog stats
POST /api/log/recent/delete  delete one recent entry
POST /api/log/recent/clear   clear the recent list
```

System & power

```
GET/POST /api/power-switch   GPIO power
POST /api/gpio-config        set GPIO pin
POST /api/auto-power-off     idle auto-off
GET/POST /api/serial-config  serial port/baud
GET/POST /api/callsign /theme
GET  /api/system             Pi host metrics
GET/POST /api/update         GitHub self-update
```

12 Troubleshooting

- No CAT / 'radio offline': check the serial port and baud in Settings; the FTDI cable must be on /dev/ttyUSB0 (or set the right port).
- No audio: ensure HTTPS, click CONNECT and allow the mic; check the USB card and its mixer levels (playback drives the radio mic on TX).
- PWA won't install / theme not switching on a phone: the certificate is not trusted — install the Root CA (chapter 9).
- RX filter only on one channel: reconnect audio (Disconnect/Connect) to renegotiate Opus to mono.
- Decoders need a clean signal; tune levels and (for RTTY) the mark tone.

13 Security

LAN-only by design; there is no authentication. Do not expose the port directly to the internet — use a VPN (WireGuard/Tailscale) or a reverse proxy with TLS + auth. The Root CA private key never leaves the Pi.

14 Credits & License

Kenwood PC protocol docs: LA3QMA/TM-V71_TM-D710-Kenwood. Built with aiortc (WebRTC/Opus) and sounddevice. Fonts: Saira, IBM Plex Mono, DSEG (7-segment), Neuropol (title). See the repository for license details.